

Reviewer Pack — Secant Variety Dimension Lower-Bounds Disjointness Communica...

Entry 9499045b5ede · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 00:38:21 UTC

Secant Variety Dimension Lower-Bounds Disjointness Communication Complexity

Entry ID: 9499045b5ede **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 00:38:21 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry of Secant Varieties **Field B** (complexity object): Randomized Communication Complexity of DISJOINTNESS

Statement:

For the communication matrix M of the DISJOINTNESS problem on n elements, the dimension of the secant variety $\text{sec}(M)$ satisfies $\dim(\text{sec}(M)) \geq c \cdot n$ for some absolute constant $c > 0$. This implies $R(\text{DISJ}_n) = \Omega(n)$ via the dimension-communication complexity duality conjecture.

Rationale (proposer’s reasoning):

Secant variety dimension captures the ‘spread’ of matrix rank in projective space. For DISJOINTNESS, the sparse structure of M forces its secant variety to have high dimension, which correlates with the need for $\Omega(n)$ communication to distinguish disjointness queries.

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d72ae361f3b3ca8f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_disjointness_matrix(n):
    matrix = [[0] * (2**n) for _ in range(2**n)]
    for i in range(n):
        row, col = random.randint(0, 2**n - 1), random.randint(0, 2**n - 1)
        while matrix[row][col] == 1:
            row, col = random.randint(0, 2**n - 1), random.randint(0, 2**n - 1)
        matrix[row][col] = 1
    return matrix

def rank(matrix):
    n = len(matrix)
    augmented_matrix = [row + [0] for row in matrix]
    for i in range(n):
        augmented_matrix[i][i+n] = 1

    def swap_rows(a, b):
        augmented_matrix[a], augmented_matrix[b] = augmented_matrix[b], augmented_matrix[a]

    def scale_row(row, scalar):
        augmented_matrix[row] = [scalar * x for x in augmented_matrix[row]]

    def add_scaled_row(a, b, scalar):
        augmented_matrix[a] = [a + scalar * b for a, b in zip(augmented_matrix[a], augmented_matrix[b])]

    for i in range(n):
        if augmented_matrix[i][i] == 0:
            for j in range(i+1, n):
                if augmented_matrix[j][i] != 0:
                    swap_rows(i, j)
                    break
            else:
                continue

        scale_row(i, 1 / augmented_matrix[i][i])

        for j in range(n):
```

```

        if i != j:
            add_scaled_row(j, i, -augmented_matrix[j][i])

rank = sum(1 for row in augmented_matrix[:n] if any(x != 0 for x in row))
return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    matrix = generate_disjointness_matrix(n)
    secant_dimension = rank(matrix) - 1
    conjecture_holds = secant_dimension >= 0.2 * n
    counterexample = "" if conjecture_holds else f"dim(sec(M))={secant_dimension}, expected ≥{0.2*n}

    return {
        "metric_name": "secant_dimension",
        "metric_value": secant_dimension,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE no_counterexamples")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3087e973.py", line 82, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3087e973.py", line 63, in run_trial
    matrix = generate_disjointness_matrix(n)
    ~~~~~^~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3087e973.py", line 19, in generate_disj
    matrix = [[0] * (2**n) for _ in range(2**n)]
    ~~~~~^~~~~~

```

MemoryError

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with MemoryError, preventing evaluation of the conjecture’s validity | next: Optimize matrix generation for sparse representations or use incremental testing with smaller n

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	106973
2	propose	ollama_remote	qwen3:8b	0	0	72502
3	preregistration	ollama_remote	qwen3:8b	0	0	19995
4	novelty	ollama_remote	qwen3:8b	0	0	16583
5	novelty	ollama_remote	qwen3:8b	0	0	11715
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10340
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10228
8	judge	ollama_remote	qwen3:8b	0	0	56997

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 305334 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/9499045b5ede.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/9499045b5ede.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/9499045b5ede.tar.gz (if generated)